Introduction to Tuples:

Definition:

Python tuples are **a type of data structure that is very similar to lists**. The main difference between the two is that tuples are immutable, meaning they cannot be changed once they are created. This makes them ideal for storing data that should not be modified, such as database records.

```
Syntax:
my_tuple = (element1, element2, element3, ...)
person_info = ('John', 25, 'Male')
```

2. Tuples with Built-in Functions:

Tuples can be used with various built-in functions in Python.

a. `len()` function:
Returns the number of elements in the tuple.

Syntax:

```
length = len(my_tuple)
```

Example:

```
person_info = ('John', 25, 'Male')
length = len(person_info)
# length will be 3
```

b. `count()` method:
Returns the number of occurrences of a specified value.

Syntax:

```
occurrences = my_tuple.count(value)
```

Example:

```
numbers = (1, 2, 3, 1, 4, 1)
count_of_ones = numbers.count(1)
# count_of_ones will be 3
```

c. `index()` method:
Returns the index of the first occurrence of a specified value.

Syntax:

```
index = my_tuple.index(value)
```

Example:

```
numbers = (1, 2, 3, 4, 3, 5)
index_of_three = numbers.index(3)
# index_of_three will be 2
```

3. Tuple Operations:

a. Concatenation:
Combine two tuples to create a new tuple.

Syntax:

```
new_tuple = tuple1 + tuple2
```

Example:

```
tuple1 = (1, 2, 3)
tuple2 = ('a', 'b', 'c')
result_tuple = tuple1 + tuple2
# result_tuple will be (1, 2, 3, 'a', 'b', 'c')
```

b. Repetition:
Repeat a tuple for a specified number of times.

Syntax:

```
new_tuple = my_tuple * n
```

Example:

```
numbers = (1, 2, 3)
repeated_numbers = numbers * 2
# repeated_numbers will be (1, 2, 3, 1, 2, 3)
```

c. Membership Test:
Check if an element is present in the tuple.

Syntax:

```
is_present = element in my_tuple
```

Example:

```
fruits = ('apple', 'banana', 'orange')
is_apple_present = 'apple' in fruits
# is_apple_present will be True
```